

Trabajo Práctico N.º 3

Patrones de Diseño Creacionales y Estructurales

Facultad	Facultad de Ingeniería - UNSTA
Carrera	Ingeniería Informática
Materia	Programación II
Profesor	Ing. Tulio Ruesjas Martín
Alumno	Nicolas Miskevish
Tecnología	Java 25 - IntelliJ IDEA
Tema	Singleton, Factory Method, Abstract Factory, Adapter, Decorator y Facade
Fecha	Septiembre de 2026

Objetivo general

Analizar e implementar soluciones de software mantenibles, escalables y desacopladas mediante patrones de diseño, representando además su estructura mediante diagramas UML.

Introducción

Este informe documenta seis patrones de diseño solicitados en el Trabajo Práctico N.º 3. Cada patrón se implementó como un ejercicio independiente en Java 25 y se organizó en un paquete propio. Los ejemplos fueron elegidos fuera del ámbito de bares, cafés, restaurantes y comida, para diferenciarlos de los ejemplos utilizados durante las clases.

Para cada ejercicio se presenta el problema, la justificación del patrón elegido, la función de las clases e interfaces, el mecanismo de funcionamiento, un fragmento de código representativo y el resultado de la ejecución. Las clases Main se utilizan únicamente como clientes de prueba para demostrar el comportamiento de cada patrón.

1. Singleton

Planteo del escenario

Se plantea una configuración global de un videojuego que almacena el volumen, la dificultad y el idioma. Distintas partes del juego necesitan consultar o modificar esos valores, pero no conviene que cada una cree su propia configuración porque podrían existir datos contradictorios.

Justificación técnica

Singleton es apropiado porque garantiza que exista una única instancia de ConfiguracionJuego durante la ejecución. Todas las referencias obtenidas mediante getInstance() apuntan al mismo objeto y, por lo tanto, comparten el mismo estado.

Clases e interfaces utilizadas

Elemento	Tipo	Responsabilidad
ConfiguracionJuego	Clase	Mantiene la única instancia y almacena volumen, dificultad e idioma.
MainSingleton	Clase cliente	Solicita la instancia más de una vez y comprueba que ambas referencias apuntan al mismo objeto.

Funcionamiento de la implementación

La clase posee un atributo static llamado instancia. Al ser estático pertenece a la clase y no a un objeto particular.

El constructor es private. Esto impide que otras clases creen objetos mediante new ConfiguracionJuego().

El método getInstance() verifica si la instancia todavía no existe. En ese caso la crea; en las siguientes llamadas devuelve exactamente la misma referencia.

La prueba modifica el estado desde una referencia y lo consulta desde otra. Como ambas apuntan al mismo objeto, los valores coinciden y la comparación con == devuelve true.

Código clave del patrón

```
private static ConfiguracionJuego instancia;

private ConfiguracionJuego() { ... }

public static ConfiguracionJuego getInstancia() {
    if (instancia == null) {
        instancia = new ConfiguracionJuego();
    }
    return instancia;
}
```

Resultado esperado de la prueba

```
Volumen desde configuracion1: 80
Volumen desde configuracion2: 80

¿Son la misma instancia?
true
```

2. Factory Method

Planteo del escenario

En un videojuego se deben generar distintos tipos de enemigos, en este caso Zombie y Robot. El código cliente necesita solicitar enemigos sin quedar acoplado a la creación concreta de cada clase.

Justificación técnica

Factory Method permite delegar la creación del producto a subclases especializadas. CreadorEnemigo declara el método crearEnemigo(), mientras que CreadorZombie y CreadorRobot deciden qué objeto concreto construir.

Clases e interfaces utilizadas

Elemento	Tipo	Responsabilidad
Enemigo	Interface	Define la operación atacar() que deben implementar todos los enemigos.
Zombie	Clase	Producto concreto que implementa Enemigo.
Robot	Clase	Producto concreto que implementa Enemigo.
CreadorEnemigo	Clase abstracta	Declara el Factory Method crearEnemigo() y utiliza el producto mediante la interfaz.
CreadorZombie	Clase	Sobrescribe crearEnemigo() y devuelve un Zombie.
CreadorRobot	Clase	Sobrescribe crearEnemigo() y devuelve un Robot.
MainFactoryMethod	Clase cliente	Crea los creadores concretos y ejecuta generarAtaque().

Funcionamiento de la implementación

El cliente trabaja con el tipo abstracto CreadorEnemigo. La clase base no conoce el producto concreto que se instanciará.

Cada subclase implementa crearEnemigo(): CreadorZombie devuelve new Zombie() y CreadorRobot devuelve new Robot().

El método generarAtaque() puede operar con cualquier objeto que implemente Enemigo, por lo que la lógica general queda desacoplada de las clases concretas.

Si en el futuro se agrega un nuevo enemigo, se puede añadir un nuevo producto y un nuevo creador sin cambiar la lógica ya existente.

Código clave del patrón

```
public abstract class CreadorEnemigo {
    public abstract Enemigo crearEnemigo();

    public void generarAtaque() {
        Enemigo enemigo = crearEnemigo();
        enemigo.atacar();
    }
}
```

Resultado esperado de la prueba

Creando enemigo tipo Zombie:
El zombie ataca cuerpo a cuerpo.

Creando enemigo tipo Robot:
El robot dispara con su arma láser.

3. Abstract Factory

Planteo del escenario

Una aplicación gráfica necesita utilizar componentes coherentes con el sistema operativo. Para Windows se requieren un botón y una ventana con estilo Windows; para Linux se necesita la familia equivalente con estilo Linux.

Justificación técnica

Abstract Factory es adecuado porque el problema no consiste en crear un único objeto, sino una familia completa de objetos relacionados. FabricaInterfaz establece qué productos debe poder crear cualquier fábrica y las fábricas concretas producen componentes compatibles entre sí.

Clases e interfaces utilizadas

Elemento	Tipo	Responsabilidad
Boton	Interface	Producto abstracto para botones.
Ventana	Interface	Producto abstracto para ventanas.
BotonWindows	Clase	Implementación Windows de Boton.
VentanaWindows	Clase	Implementación Windows de Ventana.
BotonLinux	Clase	Implementación Linux de Boton.
VentanaLinux	Clase	Implementación Linux de Ventana.

FabricaInterfaz	Interface	Fábrica abstracta: declara crearBoton() y crearVentana().
FabricaWindows	Clase	Crea la familia BotonWindows + VentanaWindows.
FabricaLinux	Clase	Crea la familia BotonLinux + VentanaLinux.
MainAbstractFactory	Clase cliente	Prueba ambas familias de productos.

Funcionamiento de la implementación

Boton y Ventana representan los tipos abstractos de productos.

FabricaInterfaz declara dos operaciones de creación: crearBoton() y crearVentana().

FabricaWindows implementa ambas operaciones devolviendo productos Windows; FabricaLinux hace lo mismo con productos Linux.

El cliente puede trabajar contra las interfaces FabricaInterfaz, Boton y Ventana, sin necesitar conocer los detalles internos de los productos concretos.

Código clave del patrón

```
public interface FabricaInterfaz {
    Boton crearBoton();
    Ventana crearVentana();
}

public class FabricaWindows implements FabricaInterfaz {
    public Boton crearBoton() { return new BotonWindows(); }
    public Ventana crearVentana() { return new VentanaWindows(); }
}
```

Resultado esperado de la prueba

```
INTERFAZ WINDOWS
Mostrando botón estilo Windows.
Mostrando ventana estilo Windows.

INTERFAZ LINUX
Mostrando botón estilo Linux.
Mostrando ventana estilo Linux.
```

4. Adapter

Planteo del escenario

La aplicación trabaja con temperaturas expresadas en grados Celsius, pero se debe reutilizar un sensor existente que devuelve valores en Fahrenheit. Las interfaces son incompatibles y no se desea modificar la clase del sensor viejo.

Justificación técnica

Adapter permite envolver el sensor existente y presentar hacia la aplicación la interfaz que esta espera. AdaptadorTemperatura recibe un SensorFahrenheit y traduce su resultado a Celsius.

Clases e interfaces utilizadas

Elemento	Tipo	Responsabilidad
SensorTemperatura	Interface	Interfaz objetivo esperada por la aplicación; expone obtenerTemperaturaCelsius().
SensorFahrenheit	Clase	Componente existente que devuelve la temperatura en Fahrenheit.
AdaptadorTemperatura	Clase	Implementa SensorTemperatura y contiene una referencia al SensorFahrenheit.
MainAdapter	Clase cliente	Utiliza el sensor a través de la interfaz compatible.

Funcionamiento de la implementación

SensorFahrenheit continúa funcionando sin modificaciones y devuelve 86 °F.

AdaptadorTemperatura implementa SensorTemperatura, por lo que puede utilizarse donde el sistema espera dicha interfaz.

El adaptador solicita el valor Fahrenheit al objeto adaptado y aplica la fórmula $(F - 32) \times 5 / 9$.

El cliente recibe 30 °C sin tener que conocer el mecanismo de conversión ni cambiar el sensor original.

Código clave del patrón

```
public class AdaptadorTemperatura implements SensorTemperatura {
    private SensorFahrenheit sensorFahrenheit;

    @Override
    public double obtenerTemperaturaCelsius() {
        double fahrenheit = sensorFahrenheit.obtenerTemperaturaFahrenheit();
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

Resultado esperado de la prueba

```
Temperatura original: 86.0 °F
Temperatura adaptada: 30.0 °C
```

5. Decorator

Planteo del escenario

Un personaje de videojuego comienza como Guerrero con una defensa base. Se desea agregar equipamiento, como armadura y escudo, y permitir combinaciones sin crear una subclase diferente para cada posible conjunto de accesorios.

Justificación técnica

Decorator permite agregar responsabilidades dinámicamente envolviendo un objeto con otros objetos que comparten la misma interfaz. Cada decorador conserva el comportamiento anterior y suma una nueva característica.

Clases e interfaces utilizadas

Elemento	Tipo	Responsabilidad
Personaje	Interface	Define obtenerDescripcion() y obtenerDefensa().
Guerrero	Clase	Componente concreto con defensa base 10.
PersonajeDecorator	Clase abstracta	Implementa Personaje y mantiene una referencia protegida a otro Personaje.
ConArmadura	Clase	Decorator concreto que agrega texto y 20 puntos de defensa.
ConEscudo	Clase	Decorator concreto que agrega texto y 15 puntos de defensa.
MainDecorator	Clase cliente	Construye y combina los decoradores durante la ejecución.

Funcionamiento de la implementación

Se crea un Guerrero y se guarda en una referencia de tipo Personaje.

La expresión `new ConArmadura(personaje)` envuelve al Guerrero. El decorador consulta primero al componente envuelto y luego agrega su propio comportamiento.

Posteriormente, `new ConEscudo(personaje)` envuelve al resultado anterior. Por eso se forma una cadena: `ConEscudo → ConArmadura → Guerrero`.

La defensa final es la suma del comportamiento de cada nivel: $10 + 20 + 15 = 45$.

Código clave del patrón

```
Personaje personaje = new Guerrero();
personaje = new ConArmadura(personaje);
personaje = new ConEscudo(personaje);

// Cada decorador suma comportamiento al objeto envuelto.
```

Resultado esperado de la prueba

```
PERSONAJE ORIGINAL
Equipamiento: Guerrero
Defensa: 10

PERSONAJE CON ARMADURA
Equipamiento: Guerrero + Armadura
Defensa: 30

PERSONAJE CON ARMADURA Y ESCUDO
Equipamiento: Guerrero + Armadura + Escudo
Defensa: 45
```

6. Facade

Planteo del escenario

Para iniciar una computadora deben coordinarse varios subsistemas: fuente de energía, sistema operativo y conexión de red. Si el cliente tuviera que controlar cada componente por separado, necesitaría conocer demasiados detalles del sistema.

Justificación técnica

Facade proporciona una interfaz simplificada sobre varios subsistemas. ComputadoraFacade centraliza la secuencia correcta de encendido y apagado, reduciendo el acoplamiento del cliente con las clases internas.

Clases e interfaces utilizadas

Elemento	Tipo	Responsabilidad
FuenteEnergia	Clase	Enciende y apaga la fuente de energía.
SistemaOperativo	Clase	Inicia y cierra el sistema operativo.
ConexionRed	Clase	Conecta y desconecta la red.
ComputadoraFacade	Clase	Agrupar los subsistemas y expone <code>iniciarComputadora()</code> y <code>apagarComputadora()</code> .
MainFacade	Clase cliente	Utiliza solamente la fachada para controlar la computadora.

Funcionamiento de la implementación

ComputadoraFacade crea y conserva los objetos FuenteEnergia, SistemaOperativo y ConexionRed.

`iniciarComputadora()` ejecuta la secuencia fuente → sistema operativo → red y presenta al cliente una sola operación.

`apagarComputadora()` realiza la secuencia inversa: desconecta la red, cierra el sistema operativo y apaga la fuente.

MainFacade no necesita conocer cómo se coordinan los subsistemas; sólo interactúa con la fachada.

Código clave del patrón

```
ComputadoraFacade computadora = new ComputadoraFacade();
computadora.iniciarComputadora();
computadora.apagarComputadora();
```

Resultado esperado de la prueba

```
Iniciando computadora...
Fuente de energía encendida.
Sistema operativo iniciado.
Conexión a Internet establecida.
Computadora lista para usar.
```

```
Apagando computadora...
Conexión a Internet finalizada.
Sistema operativo cerrado.
Fuente de energía apagada.
Computadora apagada.
```

7. Diagrama UML general

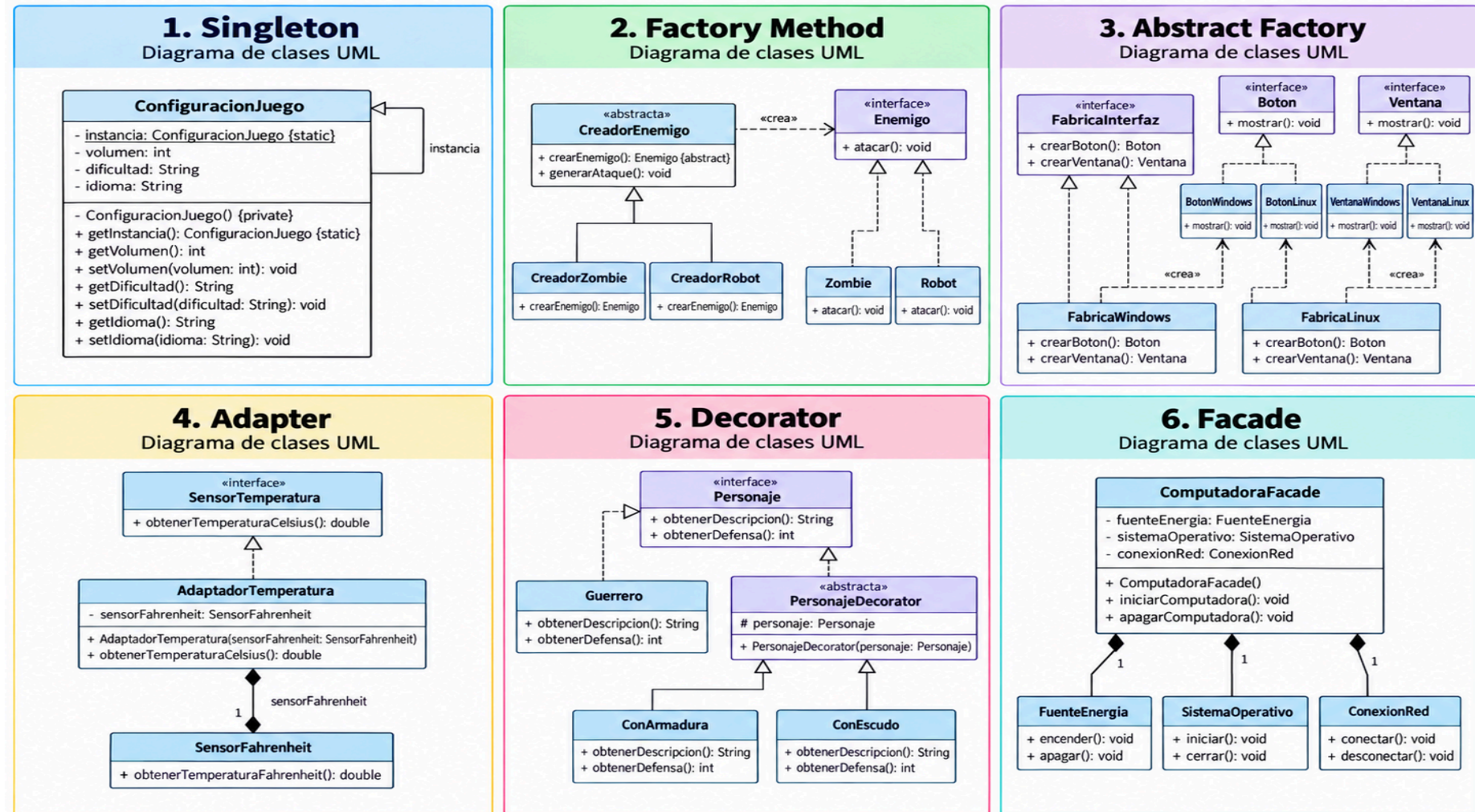


Figura 1. Resumen UML de los seis patrones implementados.

Leyenda UML: $\triangle$ línea continua = herencia/generalización | $\triangle$ línea discontinua = implementación de interfaz/realización | $\blacklozenge$ = composición | $\rightarrow$ o línea discontinua = uso, dependencia o creación | 1 = multiplicidad de una instancia

8. Explicación de las relaciones del UML

En UML, las líneas y flechas no representan únicamente conexiones visuales: indican qué responsabilidad existe entre los elementos del diseño. En este trabajo se utilizan principalmente las siguientes relaciones:

Relación	Cómo se reconoce	Significado en este TP
Generalización / herencia	Línea continua con triángulo vacío hacia la clase padre.	Una clase especializada hereda de una clase más general. Ej.: CreadorZombie → CreadorEnemigo y ConArmadura → PersonajeDecorator.
Realización / implementación	Línea discontinua con triángulo vacío hacia la interfaz.	Una clase concreta se compromete a implementar los métodos definidos por una interfaz. Ej.: Robot → Enemigo y AdaptadorTemperatura → SensorTemperatura.
Composición	Línea con rombo negro en el extremo del objeto que contiene.	El objeto contenedor crea o administra fuertemente a sus componentes. Ej.: ComputadoraFacade administra FuenteEnergia, SistemaOperativo y ConexionRed.
Asociación	Línea continua simple.	Una clase conoce o mantiene una referencia a otra. En Adapter, el adaptador mantiene una referencia al sensor Fahrenheit.
Dependencia / creación	Flecha o línea discontinua de uso, a veces acompañada por «crea».	Una clase utiliza otro tipo de forma puntual o lo construye como resultado de una operación. Es común en Factory Method y Abstract Factory.
Multiplicidad	Número situado junto a una relación, por ejemplo 1.	Indica cuántas instancias participan en la relación. Un 1 representa una única instancia asociada.